Mathematics and Computer Science

Software Engineering Non-Exam Based Assessment

Cover Sheet

Term Project Part 1

Systems Request, Business Case, and Feasibility Report

Course Name	Software Engineering	Course Code	CSCI 4153

This assessment will assess the following Course learning outcomes:

	CLO1	CLO2	CLO3	CLO4	CLO5	CLO6

Academic Honesty Statement
In accordance with SAU policy and Academic Honesty
• Students are required to refrain from all forms of academic dishonesty as defined and explained in SAU procedures and directions from SAU personnel.
• A student found guilty of having committed acts of academic dishonesty may be subject to one or more of the disciplinary measures as outlined in Articles of the Student and Academic Regulations.

	Logan Barnett
	Hannah Harcrow
	Marah Starks
	Felix Waller

Contents

Introduction to Systems Request ……...3

Business Background ……...……...5

Justification of Request ……...5

Description of the Proposed Solution …….6

Team Charter ……...8

Cost estimates (Total Cost of Ownership) ……..9

Benefits …….10

Feasibility Report …….11

Requirements Engineering ……...14

User Stories and Product Backlog ……16

Justification of Request …….…18

Security…….….20

Developed Software……...…21

Testing Stages and Testing Techniques……..…23

Samples of how Testing Can Be Achieved……26

Use Case Diagram & Use Case Description …….28

Design Pattern……29

Design Pattern Protoype……30

Introduction to Systems Request

System Request	
Date: 1/20/2026	**Department:** Data Management
Submitted by: John Bottles (Manager of DMV)	**Address:** 135 Jackson St SW, Camden, AR 71701
Title: AI Managerial Supervisor	**Email:** Johnbottles@camdendmv.org
Request For: New System	**Urgency:** Moderate Priority
Description of Request: We need a manager AI that can monitor our employees to ensure they are on-task, and prevent mistakes. The manager AI will compare the information on clients that employees enter with	

System Request	
that of which we have on the company's database, and inform the employee and manager if there is a mismatch. The AI will tally up how many times mismatched data has been entered by each employee to flag them for investigation by the manager; the manager can remove "false-alarm" flags. The AI will also detect what software the employees are using and inform the manager if they are off-task (which will also be tallied). The AI will time the employees on tasks to calculate average completion times, and which employees are working in the most timely manner.	
Reasons for Request: ● Increased employee mistakes, which slows down workflow. ● Employees playing games on their computers during work. ● It is difficult for the manager to monitor the computers of each employee, when the manager has other responsibilities.	
Projected Budget: $20,000 + $1550/month	
Client Signature: John Bottles	**Date:** 1/20/2026
Development Team Signatures: Felix Waller Venus Starks Hannah Harcrow	**Date:** 1/20/2026

Business Case	
Project Title: AI Managerial Supervisor for DMV	
Business: The Camden, AR DMV deals with matters related to licenses and motor vehicle legality. The employees enter sensitive client data, and the data must be correct for later reference of the clients. The employees must be monitored to ensure they are dealing with the sensitive information with the utmost importance, and not straying from task.	
Problem: Employee productivity seems to be at an all-time low due to employees entering in mistakes and doing off-task activities on computers. The increase of mistakes and lack of focus from the employees makes the office much slower, since client data is not documented properly and cannot confidently be referenced. The manager cannot keep a keen eye on every employees' computer, while still completing his/her tasks simultaneously.	
Proposed Solution: A manager AI that can monitor our employees to prevent mistakes would be the ideal solution to our problem. The manager AI will compare employee-entered data with that on-file for possible mistakes, and monitor employee screen activity to make sure they are on-task.	
Estimated Time of Completion: 5 Months	**Investment Amount:** $20,000 + $1550/month
Net Monthly Profit: $2,850	**Payback Period:** 7.018 months
Yearly ROI: 78%	
Risks of Project: ● Potential breach of employee privacy ● Similar software releasing by other companies	
Risks of Continuing Without the Project: ● Confusing and unreliable data on clients ● Slow wait times ● Stress from high workloads ● Customer loss	
Success Measurements: ● Decrease in time needed for employees to complete tasks ● Increase in positive customer reviews after launch	

Business Background

Camden DMV (Department of Motor Vehicles) in Camden, AR aids those who are drivers/preparing to become drivers. Camden DMV supplies its customers with driver examinations, driver licenses, driving records, driving violation handling, vehicle registration, license plates, disability tags/plates, and vehicle tax services. The Camden DMV also benefits those who are not drivers by issuing them state IDs. Camden DMV's employees are responsible for completing all the above tasks in a timely manner, which often involves inputting sensitive customer data into the DMV's database– and retrieving that data– accurately.

Justification of Request

Basically, the whole point of this project is to update how we track employee's performance. Traditional managers just can't keep pace with real-time data, so we're trying to close those holes. Putting an AI manager in as HR's dedicated sidekick lets us watch things constantly - for example how fast people process tasks or how accurate their documentation is - way beyond what any human could do around the clock. It spots exactly where someone keeps getting hung up on a task and flags it right away for feedback, no more waiting around for the monthly or yearly sit-down. The main idea is to make everything run smoother: dump all the performance numbers into one spot, sort them automatically, and shoot the summary up the chain in minutes instead of days.

To ensure the comfortability of our users, we must highlight that our software does not have access to cameras or microphones. Algorithms have zero emotional intelligence. They see a performance dip and don't get why, maybe the person's going through something rough at home, or maybe it's just ADHD making focus tricky that day. If we lean too hard on pure numbers, the whole office might start feeling cold. To fix this, we are proposing a leniency feature so the AI

doesn't just flag someone for having a single bad day. Getting this balance right could actually create a new kind of AI, one that's there to help people do their jobs better and keep things fair, without ever trying to take over from the actual humans in charge.

Description of the Proposed Solution

Our team came up with this AI manager system to help out at a place like the DMV. It keeps tabs on how employees handle customers with license renewals, registrations, and all that, without the AI ever chatting with anyone directly. The goal is to give real managers solid numbers on speed, accuracy, and overall work so they can make fair calls on performance. We split the whole thing into hardware, software, processes, data, and the people side. It feels like a solid upgrade that still leaves room for human judgment.

On the hardware side, we must highlight that our software will not have access to the camera or the microphone on the device. Our software will only be watching the data being input by the employees and comparing it to its original database provided by the company. Everything runs on a basic central computer or small server rack in the back office. Nothing crazy expensive, just enough to keep the software running smoothly without lagging.

The software piece would be built around a Python-based AI manager that uses recurrent neutral networks for tracking sequences of task times and document checks. It runs a simple classification to sort workers into good, okay, or bad categories based on timers and mistake tallies. We created and tied a Google Sheets database for customer information for the AI to use. Reports can be accessed easily by a clean and easy to navigate dashboard. That can only be accessed by managers. A leniency switch will be added to the code so it does not hammer an employee having a single bad day.

The processes work like this: the AI starts a timer when the employee pulls up a customer file, analyzes the new data to the old data, tallies mistakes, writes it on that particular employee's report, and stops the timer once the employee closes out the customer's file. It counts how many people get helped in a shift and scans whether the paperwork got filled out right. If mistakes pile up or the task seems to be taking longer than expected, it logs it and shoots a quick summary over to the company's manager within a reasonable timeframe -about 30 seconds to a minute. The manager reviews the numbers, adds any notes about if that particular employee has ADHD/ADD etc, and decides next steps. Every week it bundles everything into one report file so nothing gets lost in separate documents.

Data flows from a couple spots. Employee logs store timestamps, customer count, and a running mistake counter. The customer side sits in a simple spreadsheet with zero or one flags for background issues like DUIs so checks stay fast. Developers feed in labeled training examples at first to teach the AI what timely and accurate information looks like. Managers can jump in and correct wrong calls - which will allow updates to our AI model over time. All the sensitive information - such as the employee's reports and the customer's data - stays locked behind permissions so only those who have administrative access can see it.

People stay at the center of it. Real managers and HR get the reports and keep full control over promotions, write-ups, etc. Employees just do their normal jobs, knowing the system watches quietly in the background. Our team handles the initial setup and training, and we made sure the whole thing acts like an assistant, not a replacement. The human manager always has the final say. Our goal is to ensure that our software will not replace or take over any human jobs. It's a tool that will help and ensure efficiency within the workplace.

Team Charter

Project Name: AI Managament
Vision & Mission: To create an AI management system that helps improve the efficiency, cost, and performance in a workplace.
Success Criteria:
Project Team: Manage IQ

Names:	Role:	
Marah Starks	Student	
Hannah Harcrow	Student	
Felix Waller	Student	
Logan Barnett	Student	

Rules of Behaviour:	
Communications:	Discord Server

Approved by the Course Instructor:	Date: 2/2/2026

Cost Estimates (Total Cost of Ownership)

Initial Costs	
Source	Cost Estimate
Equipment For AI software hosting	$5,000.00
Observation Tools (Software)	$2,000.00
Developmental Costs	
Source	Cost Estimate
Developers at Median rate of $65/hr each at ~200 hours of work total. (Estimations are made using AI Models and will vary for case to case)	$13,000.00
Estimated System Upfront Cost	$20,000.00
Operational Costs	
Source	Cost Estimate
Sever Maintenance	$200/month
Energy Costs	$500/month
3rd Party AI API Access	$150/month
Total Estimated Monthly Costs	$1550/month
Total Estimated Monthly Costs	$1550/month

Benefits

The company would see clear, measurable benefits from building and using this AI Manager system because it would speed up a lot of the DMV's - or any company's - slow, repetitive work. Since the AI uses supervised learning, classification, and logistic regression to check things like SSNs, background info, license status, and other DMV requirements, employees' common mistakes will be caught and notified. Even saving just a few minutes per customer adds up fast - if an employee normally handles around 30 people a day, cutting the process by 3-5 minutes each could let them help several more customers without hiring extra staff. That also means fewer mistakes and less time wasted fixing errors, which can save the office thousands of dollars a year in labor and correction costs. With AI tracking how long tasks take and how often mistakes happen, managers can also schedule staff more efficiently and avoid paying for unnecessary overtime.

Our goal with the system is to make the workplace feel more organized and fair. Instead of managers guessing who's doing well or who's falling behind, the AI gives consistent, unbiased data about how employees are performing. It doesn't judge people - it just watches and reports - so workers can trust that evaluations are based on actual patterns, not personal opinions. It also helps managers spot who needs help, who's improving, and who's overloaded. Over time, this kind of clarity can improve teamwork, customer experience, and even employee confidence, because people know exactly what's expected and how they're doing.

Feasibility Report

Economic Feasibility Analysis:

There are two primary characteristics of this system which create profit, reduced employee analysis time, and profit gained through more efficient workflow.

For estimation, consider the system to be effective enough to encourage an employee to have an extra half hour of productive work a day. For evaluating a small business of ~400 employees, this would create roughly 1000 hours of productive work per week. 1000*4 = 4000 hours of productive work per month, assuming the DMV has a lower profit rate off each employee ($11/hr). This creates $44,000 a month. In 1.24 months the initial costs would be covered. This would leave the company with $42,450 a month in extra profit due to this project. The project is considered economically feasible due to high return on investment.

Category	Value
Number of Employees	40 Employees
Estimated gained hours per capita each day	0.5 hrs
Estimated Profit per hour ($)	11 $/hr
Weeks In a Month	4 weeks
Estimated system monthly cost ($)	1550 $/mo
Estimated system upfront cost ($)	$20,000
Total extra hours/week	40 employees * 0.5 hours/day * 5 days/week = 100 hrs/week
Total extra hours/month	1000hrs/week * 4weeks/mo = 400 hrs/mo

Monthly profit ($)	4000*hrs/mo * `11 $/hr = 4400 $/mo
Net monthly profit ($)	4400 - 1550 = 2850 $/mo
Break-even time (months)	**$20,000/2850 $/mo = 7.018 months**
3rd Party AI API Access	$150/month
Total Estimated Monthly Costs	$1550/month

Operational Feasibility:

We looked at whether this AI manager system could actually fit into the daily work at the DMV in Camden, AR. The big question was how the employees and the manager would handle it once it went live. Our software only checks data input, which programs are open, and never touches cameras or microphones. This should help ease our users so that they do not feel watched. Most employees already use computers for everything, so this just adds a quiet background tool that runs automatically when they open a customer file.

The system will be pretty easy to operate day to day. When an employee starts a customer check-in, the timer kicks in by itself, compares the new info to the database, and logs any mismatches. The manager dashboard is simple, just a clean screen that shows tallies, average times, and flags. We built in the leniency switch so the AI does not fault that particular employee for one day off, which should keep things fair and stop people from getting upset. Training would only take about thirty minutes in a group session where the manager shows everyone the basic reports and what this software does. The managers can also be shown how to add notes for critical reminders of an employee like ADHD or personal issues.

We did the operational feasibility part by going back over the systems request and business case. Then our team talked about real-world situations. For example, when an employee pushes back in retaliation or does extra steps in the workflow unlike other companies. We also thought about how similar monitoring tools work in other offices.

Overall, the project looks operationally feasible. It lines up with how the DMV already runs, gives the manager real help without extra work, and would make the whole office run smoother once everyone gets used to it. The human manager still makes all the final calls, so no human jobs are getting replaced.

Technical Feasibility:

To decide whether the Manager AI project was technically feasible for the Camden DMV, and for our software engineers, we investigated the project's compatibility with existing technology, scalability, reliability, performance, and our technological expertise. There are many pieces of existing hardware geared towards the intent of AI hosting, one of the most popular options being Dell's PowerEdge servers. Dell has many different PowerEdge servers composed of different hardware for different purposes, and there are many options that would fit our requirements, such as the Dell PowerEdge T560 (Tower) and the Dell PowerEdge R760xa. The scalability, reliability, and performance of the manager AI is greatly influenced by both the coding of the AI software itself, but also the server the AI is hosted on. Given the many fitting options, the server should not have a problem with scalability, reliability, and performance–handling the needed workloads while still staying up and performing needed operations. We also ensure that our software engineers understand the structure and workings of AI, and are

confident they will be able to create efficient software, with minimal to no problems that will affect the AI's workload, ability to operate, and need to carry out required tasks.

All around, the Manager AI should be a technologically feasible project at the current time. The needed hardware resources are greatly available. Our software engineers understand the structure of AI, and have the ability to create AI through Python. Overall, scalability, reliability, and performance issues will be absent through the collaboration of both the hardware and engineers.

Requirements Engineering

Functional:

The Manager AI must be able to recognize the difference between suit behavior and unfavorable behavior in employees.

- The Manager AI must be able to view each employees' screen to ensure they are only using on-task software. Off-task software is unfavorable.
- The Manager AI must be able to compare employee-entered data with what is already on file to detect a possibility of a mistake being entered. Mistakes are unfavorable.

To track the above tasks, the Manager AI must be able to monitor and analyze multiple sets of data at once. The AI must also tally the above unfavorable behaviors in an employee information spreadsheet.

The Manager AI must be able to time each employees' "time-to-completion" for each task, and store those times on the employee database for later analysis by the real manager. The Manager UI software (that the real manager manually interacts with) must be able to pull tally

marks and times to complete tasks from the Manager AI's employee spreadsheet for each employee. The real manager must be able to reverse tallies, and mark them as false alarms (AI will never be 100% accurate). The Manager UI must be able to sort employees from worst to best (most to least) based on tally mark counts, and the Manager must also be able to search employees. The real manager must be able to view all AI-pulled results on the Manager UI.

Non-Functional

The Manager AI should create weekly reports, instead of one long report, creating a more user-friendly and less daunting approach for the real manager. The real manager should be able to access these reports from the click of a button on the Manager AI Software, instead of navigating files to find the reports. After clicking the button, the Manager AI Software should take no more than 30 seconds to 1 minute to create a report for the real manager, giving the Manager AI time to process, while not wasting the real manager's time. These reports should be printed in a format that is easily readable and comprehensible by the real manager. Past reports should have the capability of being deleted by the real manager from the Manager AI Software, so that the real manager's "workspace" (UI) is less cluttered. Even when deleted, past reports should still be archived in storage, in case they are needed to be looked back upon.

The real manager should get notifications for every alarm of suspicious employee behavior the AI assesses, so the real manager can respond in real time. With each notification the real manager receives on suspicious employee behavior, the real manager should be able to fill out feedback for the AI, so the AI can train to fit the workspace more over time.

Only authorized personnel, the real manager and software engineers, should have access to the Manager AI. The software engineers should have access to the Manager AI and Manager AI Software's code, while the real manager should have access to train and request material from the Manager AI. For convenience, the Manager AI Software should be able to host multiple users at once. The real manager could have multiple computers, there could be multiple managers, or the software engineers may need access at the same time as the manager(s).

User Stories and Product Backlog

To show the level of importance of each user story, I will color code them. Those colors are red for top priority, yellow for tasks that will need attention later, and green for tasks of least importance.

Here are our User Stories:

Product Owner: As a Director of a DMV, I want to see the status of my employees so I can see if they are working efficiently.

Main User: As an employee at a DMV, I want a way to prove to my managers that I am doing my job efficiently.

Scrum Master: As a client that goes to the DMV, I want a way where the DMV wait time is decreased significantly to 30 minutes.

Product Backlog					
T ID	Task Name	Sprint #	Assigned To	Start Date	Finish Date
00	Created a reference	00	Starks, Marah	01/22	02/03

Product Backlog					
T ID	Task Name	Sprint #	Assigned To	Start Date	Finish Date
	document to start our project called AI Bare Bones		Harcrow, Hannah		
01	Performed SWOT analysis	01	Starks, Marah Harcrow, Hannah	02/04	02/04
02	Created Functional and Non-Functional requirements	02	Starks, Marah Harcrow, Hannah Barnett, Logan Waller, Felix	02/05	02/09
03	Created final paper and assigned parts	03	Harcrow, Hannah	02/09	02/09
04	Created slide show	04	Starks, Marah	03/02	03/09
05	Wrote Benefits Section	05	Starks, Marah	03/03	03/03
06	Created Use-Case Diagram	06	Starks, Marah	03/03	03/09
07	Wrote Cost Estimates Section	07	Barnett, Logan	03/09	03/09
08	Wrote the Economic Feasibility Report	08	Barnett, Logan	03/09	03/09
09	Wrote the Use-Case Diagram section	09	Barnett, Logan	03/09	03/09
10	Wrote the System's Request section	10	Waller, Felix	03/11	04/08
11	Created the Composite Pattern	11	Starks, Marah	03/13	03/18
12	Wrote the Justification Report section	12	Harcrow, Hannah	03/18	03/18

Product Backlog					
T ID	Task Name	Sprint #	Assigned To	Start Date	Finish Date
13	Wrote the Proposed Solution section	13	Harcrow, Hannah	03/18	03/18
14	Created the Design Pattern	14	Barnett, Logan Waller, Felix	03/22	04/11
15	Wrote the Operational Feasibility Report	15	Harcrow, Hannah	04/06	04/06
16	Wrote the Justification of Request section	16	Harcrow, Hannah	04/06	04/06
17	Created the Prototype	17	Barnett, Logan Waller, Felix	04/06	04/17
18	Wrote the User Stories and Product Backlog section	18	Starks, Marah	04/06	04/21
19	Business Background section	19	Waller, Felix	04/08	04/08
20	Wrote the Technical Feasibility Report	20	Waller, Felix	04/11	04/11
21	Wrote Security section	21	Starks, Marah	04/13	04/13
22	Wrote Functional and Non-functional Requirements	22	Waller, Felix	04/13	04/16
23	Wrote Developed Software section	23	Starks, Marah	04/18	04/18
24	Wrote Testing Stages and Testing Techniques	24	Harcrow, Hannah	04/19	04/19
25	Wrote Samples of	25	Harcrow, Hannah	04/19	04/19

Product Backlog					
T ID	Task Name	Sprint #	Assigned To	Start Date	Finish Date
	how testing can be achieved				

Justification of Request

The DMV is dealing with some real problems right now that are making everything slower than it needs to be. Employees keep messing up when they enter client information - for example license numbers or registration details. Then the office has to go back and fix those errors later, which just wastes the time of the company and its clients. On top of that a bunch of workers are getting distracted on their computers or checking their phones instead of staying on task. The manager cannot keep an eye on every single computer screen while trying to do their own job at the same time. All of this adds up to lower productivity, longer customer wait times, and customer data that nobody can really trust later on.

But, there are good opportunities for the company if we fix these issues. Catching mistakes as soon as they happen and keeping people focused could let the office help more customers every day. Managers would get automatic reports showing who is working fast and accurately. So performance reviews would actually feel fair and it would be easier to make schedules.

To get a clear picture of where things stand right now, we put together a quick SWOT analysis. Here are our results:

Strengths: The DMV already has a good solid database of customer records and the staff knows the license and registration process pretty well. There are set procedures in place and most employees are comfortable using computers for their daily work.

Weaknesses: There is a high number of data entry mistakes plus people getting off task a lot and the manager is stretched way too thin to watch everyone properly. It all adds up to a slower and less accurate office.

Opportunities: Bringing in the AI supervisor would speed up how long tasks take, cut down on errors and give managers clear data to make smarter decisions. It could also make the whole workplace feel more organized and fair because everyone would know exactly how they are doing.

Threats: A few employees might still feel uneasy about being monitored even though there are no cameras involved. There is also a chance the AI could flag things by mistake if it is not set up right. And if we do not explain the system clearly to everyone it could end up hurting morale or even cause some people to leave.

When we look at all of this together, it is pretty clear why the request makes sense. The problems are real and they are costing the office time and money every day. But the opportunities are big enough that adding the AI tool should pay off quickly and help the whole DMV run better for employees, managers and customers.

Security

We cannot eliminate risks that our software will have, but we can mediate a plan to ensure that it stays secure and safe. Before establishing the software, we will do a preliminary risk, design risk, and operational risk assessment. This will inform us on what hardware systems and work environment the DMV has. That information will influence the types of attacks and security measures that we will need/do.

This software is dealing with a lot of confidential data that is being sent back and forth between the employee, the AI, and the manager. This easily leads us to having security being one of the top priorities. Since the AI is constantly collecting data like timestamps, mistakes, and work speed, we have decided to establish a strong protection. This entails practices such as encrypted storage, secure login systems, and strict access controls so only the manager or authorized staff can access the reports.

The IT team of the company will keep up these security practices, by ensuring that the AI itself stays trustworthy and accurate. That means regularly checking the system for bugs, updating it to patch vulnerabilities, and using tools that detect unfamiliar and suspicious activity. The AI will also keep a detailed log of who did what in the system. This will help the managers see who might have comprised the company or help the IT team see who infiltrated the system. By keeping the system secure and well-maintained, the AI can do its job – watching, comparing, and reporting– without risking privacy issues or giving out wrong information.

Developed Software

This section will go over the type of data and processes that we will implement in our software. We have also provided xml code to show an outline of what the code will look like.

The data in our system is the information the AI uses to compare and contrast the employee's input during each shift. I have to reiterate that our software does not use cameras or microphones. It only looks at what the employees type into the system and compares it to the company's official database. The main types of data include:

- **Employee Logs:** this includes timestamps, task durations, mistake counts, etc
- **Customer Data:** provided by the company
- **Training Data:** examples from developers showing what is "correct" and "incorrect" work looks like
- **Updated Corrections:** Used to improve the AI model over time
- **Weekly Summaries:** combined reports for managers and HR

Here is an example of xml code to show how the data would be structured:

Input XML

```xml
<DataSystem>
    <EmployeeLog>
        <Employee id="E104">
            <TasksCompleted>27</TasksCompleted>
            <AverageTaskTime>4.2</AverageTaskTime>
            <Mistakes>1</Mistakes>
            <PerformanceLevel>Good</PerformanceLevel>
        </Employee>
    </EmployeeLog>

    <CustomerDatabase>
        <Customer id="C552">
            <Name>John Doe</Name>
            <DUI_Flag>0</DUI_Flag>
            <LicenseStatus>Active</LicenseStatus>
        </Customer>
    </CustomerDatabase>

    <Security>
        <AccessLevel>ManagerOnly</AccessLevel>
        <Encryption>Enabled</Encryption>
    </Security>
</DataSystem>
```

XML Tree

object ▶ DataSystem ▶ Security ▶

```
▼ object {1}
  ▼ DataSystem {3}
    ▼ EmployeeLog {1}
      ▶ Employee {5}
    ▼ CustomerDatabase {1}
      ▶ Customer {4}
    ▼ Security {2}
        AccessLevel : ManagerOnly
        Encryption : Enabled
```

The software's process is made to be simple, fast, and non-intrusive. Here is how our process works:

1. If employee opens a customer's file, start a timer
2. If the employee enters new data on the customer's file, compare the new data to the old
3. Check for mistake imputed by the employee and take note of any mistakes on employee's file
4. When the employee closes the customer's file, end timer and take note on their file
5. If something looks unusual - such as inputting too many errors or becoming less efficient - of the employee, send a short summary to the manager
6. At the end of each week, bundle everything into one organized report for the manager

Here is an example of xml code to show how the process would be structured:

Input XML Sample

```xml
<ProcessFlow>
    <Step order="1">StartTimer</Step>
    <Step order="2">AnalyzeInput</Step>
    <Step order="3">CheckForMistakes</Step>
    <Step order="4">StopTimer</Step>
    <Step order="5">SendManagerAlert</Step>
    <Step order="6">CompileWeeklyReport</Step>
</ProcessFlow>
```

XML Tree

object ▶ ProcessFlow ▶ Step ▶ 5 ▶

```
▼ object {1}
  ▼ ProcessFlow {1}
    ▼ Step [6]
      ▼ 0 {2}
          _order : 1
          __text : StartTimer
      ▼ 1 {2}
          _order : 2
          __text : AnalyzeInput
      ▼ 2 {2}
          _order : 3
          __text : CheckForMistakes
      ▼ 3 {2}
          _order : 4
          __text : StopTimer
      ▼ 4 {2}
          _order : 5
          __text : SendManagerAlert
```

The hardware technology will be a small server or central computer in the back office that has a low cost setup. The software technology would be based on what software the company uses to store their customers' data.

Testing Stages and Testing Techniques

Checking that every piece matters when building the AI Managerial Supervisor. Since it pairs with standard tech, like timers, dashboards, and data checks, along with smart features such as spotting focused or distracted employees, catching errors through neural nets, plus sorting how well someone performs, a mixed way of testing fits best. It keeps things precise, steady, and fair while also guarding personal space by skipping any need for cameras or mics.

Testing moves through familiar V-Model phases, reshaped slightly to fit how AI supports the process.

1) Unit Testing

   Individual components are tested in isolation before integration.

   Techniques: White-box testing, boundary value analysis, and equivalence partitioning.

   Focus: Timer start/stop functionality, data comparison logic, mistake tally counter, performance classification (good/okay/bad), and a leniency toggle.

   Tools: Python's unittest framework and isolated test datasets for the AI model.

2) Integration Testing

   Models are combined and tested to ensure they interact correctly.

Techniques: Incremental (bottom-up) and big-bang integration testing.

Focus: Interaction between the timer, data comparison engine, mistake tallying, and report generation. Special attention is given to data flow between the AI and the manager dashboard, as well as integration with the Google Sheets database.

3) System Testing

The fully integrated system is tested against both functional and non-functional requirements.

Techniques: Black-box testing, end-to-end scenario testing, performance/load testing, and security testing.

Focus: Accuracy of mistake detection, on-task monitoring, average task completion time calculations, weekly report generation speed (within 30-60 seconds), and support for multiple simultaneous users.

AI-specific testing: Model robustness against noisy or varied input data.

4) Acceptance Testing

Stakeholders (DMV managers and employees) validate that the system meets real-world needs.

Techniques: User Acceptance Testing (UAT) and Alpha/Beta testing.

Focus: Usability of the manager dashboard, fairness of flagging (enabled by the leniency feature), measurable productivity improvements, and confirmation that employee privacy is protected.

AI-Specific Testing Techniques

1) *Bias and Fairness Testing:* Ensure training data is balanced and check for unintended bias in performance classification.
2) *Adversarial/Edge Case Testing:* Test scenarios such as legitimate slow days due to complex customer requests to verify the leniency feature works correctly.
3) *Model Drift Testing*: Simulate changing work patterns to ensure the AI adapts appropriately or raises proper alerts.
4) *Explainability Testing:* Confirm that managers can understand why a particular flag or classification was made.

Samples of How Testing Can Be Achieved

The following are practical examples of test cases used to verify the AI Manager Supervisor system:

Example 1: Unit Test - Mistake Detection

Test Case ID: UT - 001

Description: Verify the AI correctly identifies a data mismatch.

Input: Employee enters SSN "123-45-6789" while the database record shows "123-45-6790."

Expected Output: Mistake tally increases by 1 and a mismatch flag is generated.

Method: Run the comparison function in isolation using mock database data and assert the correct tally and flag.

Example 2: Integration Test - Timer and Reporting

Test Case ID: IT - 005

Description: Employee opens a customer file, works for 4.5 minutes, and closes it with one mistake.

Expected Output: Timer records 4.5 minutes, mistake is logged, average task time is updated, and a summary appears on the manager dashboard within 30 seconds.

Method: Simulate file open/close events and verify correct data flow between modules.

Example 3: System Test - Off-Task Detection

Test Case ID: ST - 017

Description: Employee switches to a non-work application (e.x. A game) during a customer session.

Expected Output: AI detects off-task software, logs the event, increases the off-task tally, and sends a notification to the manager.

Method: Execute the full system in a controlled environment with simulated screen activity.

Example 4: Acceptance Test - Leniency Feature

Test Case ID: UAT - 003

Description: Manager enables leniency mode for an employee having one unusually slow day.

Expected Output: No persistent negative flag is applied, manager can add a note (e.x. “Complex case, legitimate delay).

Method: DMV manager tests the prototype dashboard during a simulated workday and provides feedback on fairness.

Example 5: Security Test

Test Case ID: SEC - 001

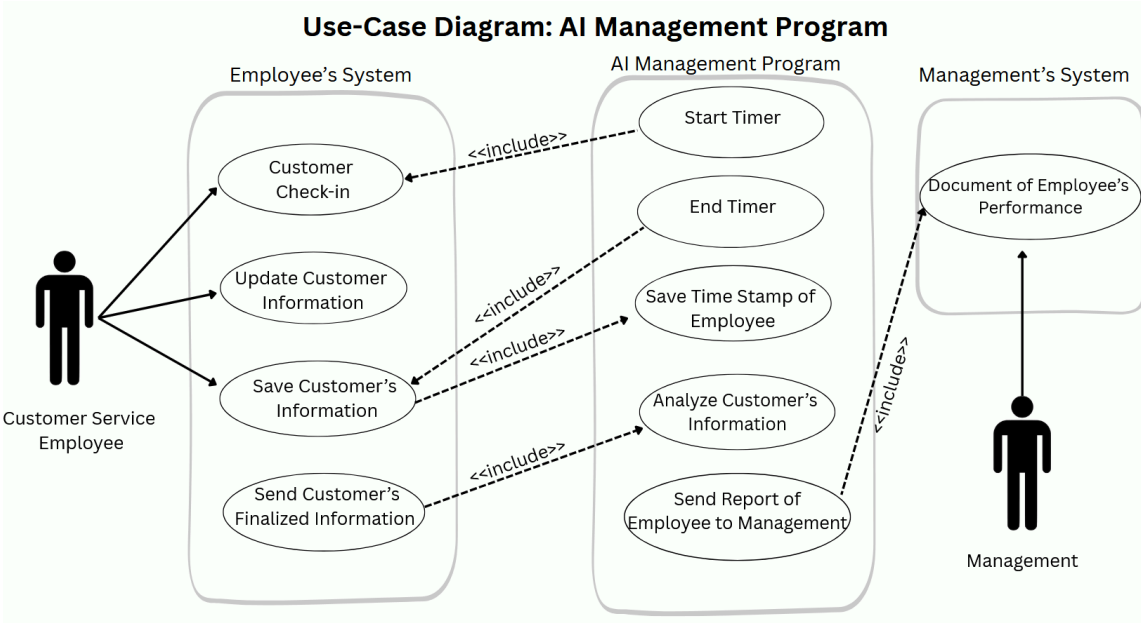
Description: Attempt unauthorized access to employee performance reports.

Expected Output: Access is denied; only authorized manager accounts can view reports.

Method: Role-based access testing with different user accounts.

After deployment, automated tests will run daily, and key metrics (mistake detection accuracy, false positive rate, and system response time) will be continuously monitored. Manager feedback will be used to retrain the AI model when needed.

Use Case Diagram & Use Case Description:



Use Case Description: Customer Check-in

Primary Actor: Customer Service Employee

Description: The Use case accounts for when an employee checks a customer in which starts a time for the customer and employee interaction time to be measured.

Trigger: An employee checks a customer in.

Relationships: Customer Service Employee and Management

Association: The actors access an Employee's system and an AI management program.

Flow of Events:

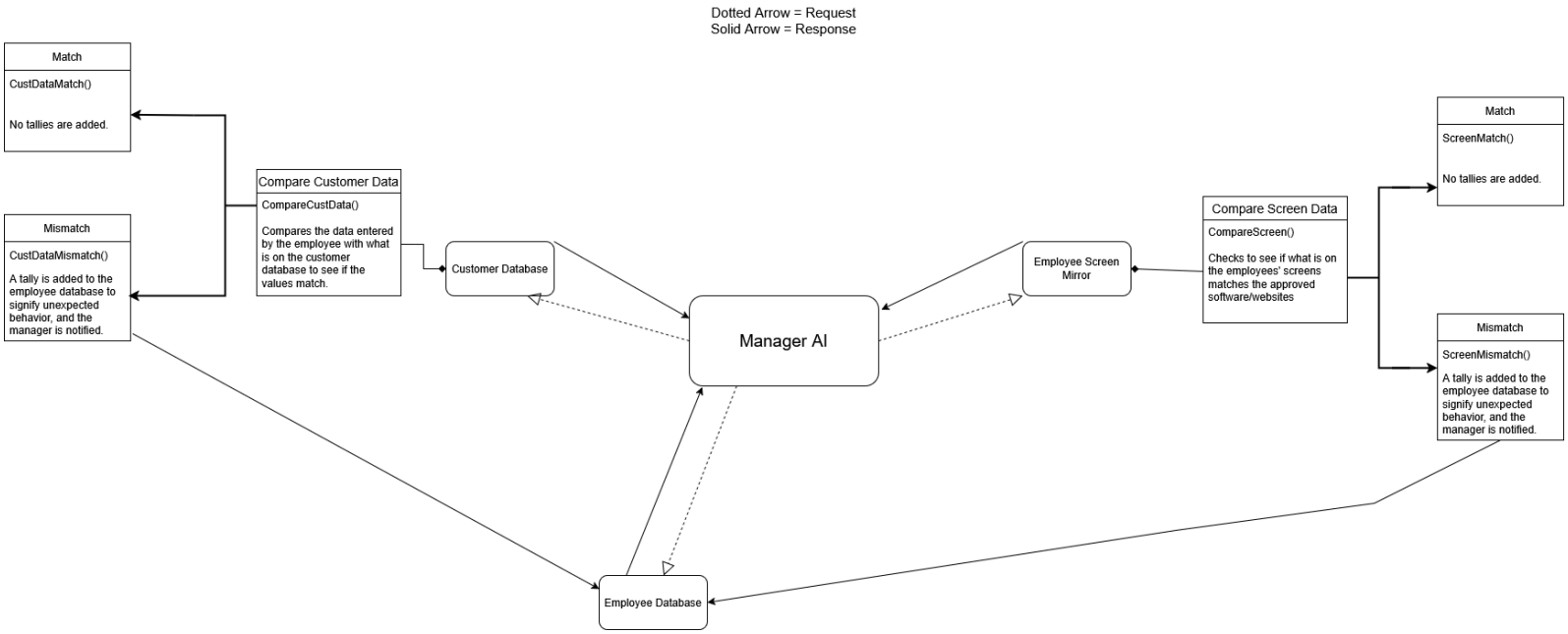
Customer Engagement — Customer begins engaging in customer service.

Check in logging — Employee Logs Customer Check in

Employee and AI system's Talk — The employee system transfers the start time of the engagement to the AI Management System.

Start Time Data Logged — The Start time data is logged into the AI System for future reference.

<u>Design Pattern Graph</u>

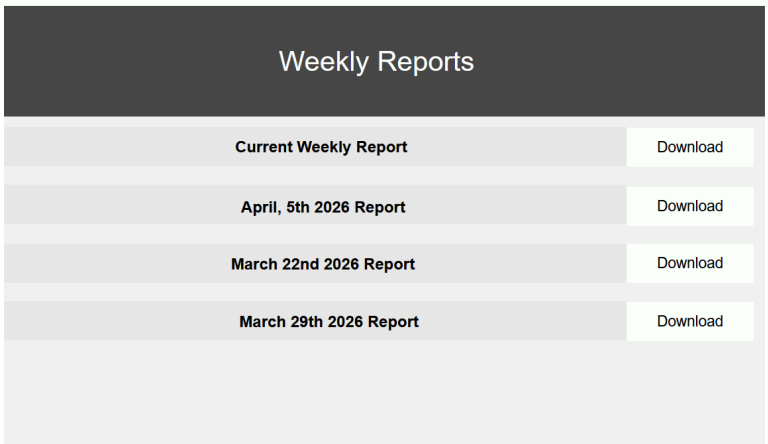


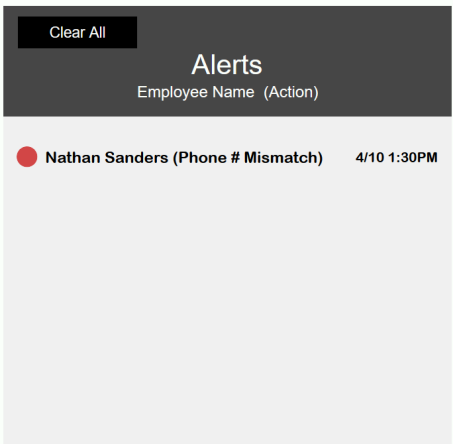
Design Pattern Prototype

Link to JustinMind Prototype

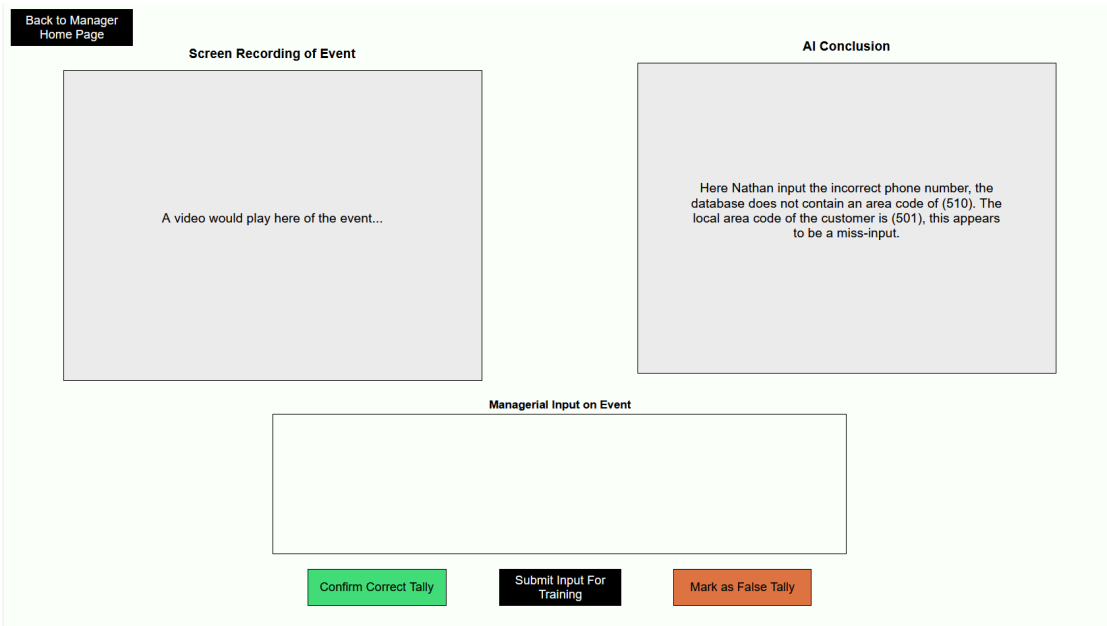
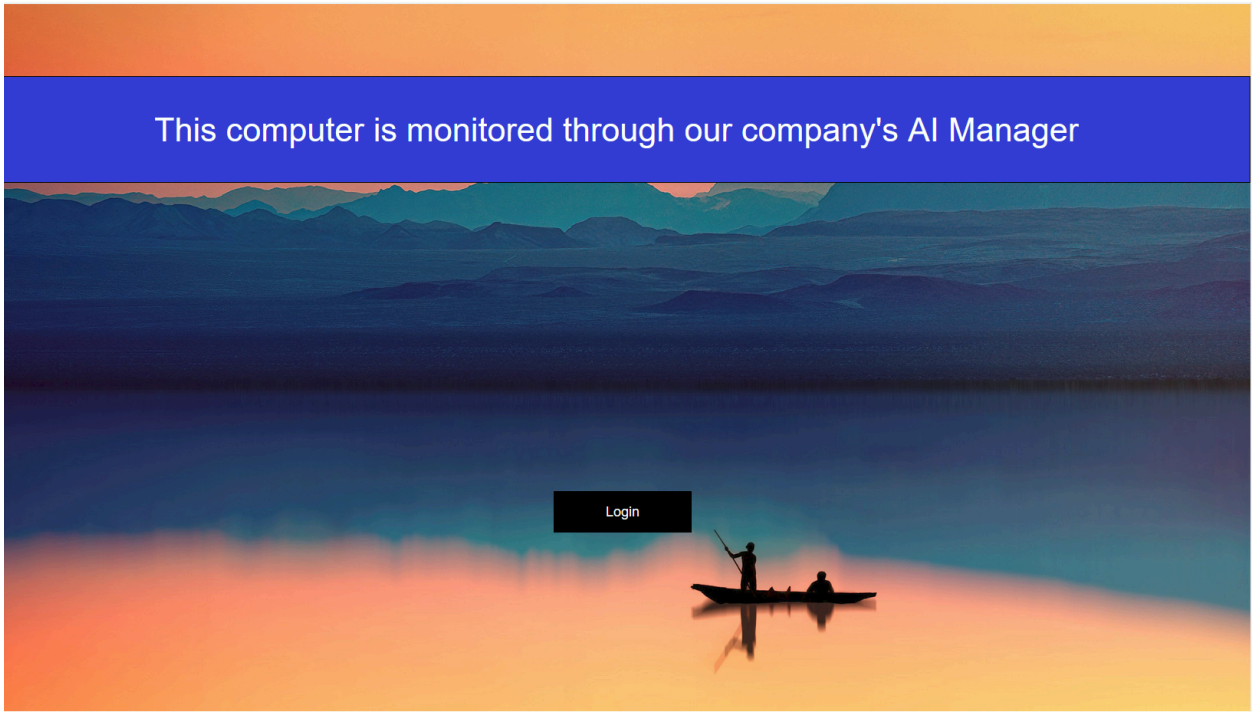
https://cloud.justinmind.com/usernote/prototype/02d9ccc7d76439f6032c249230a73eb7a757916d7e45c03443ad548d924628d4

Manager Landing Page:





Manager, Event Management Page:



A view of what the employee’s side would look like: